SALESTORM

Stage 8 – Architecture Decisions & Trade-offs

1. Problem Statement

SALESTORM must handle extreme flash-sale traffic while maintaining inventory correctness, reliable payments and scalable processing. Therefore, suitable choices must be made for database, communication, concurrency and caching.

2. ADR-001: SQL vs NoSQL

Decision: Use **SQL** for transactional data.

SQL	**NoSQL**
Strong transactions	High scalability
Strong consistency	Flexible schema
Suitable for orders, payments and inventory	Suitable for high-volume non-critical data

3. ADR-002: Synchronous vs Asynchronous

Decision: Use a Hybrid approach

- **Synchronous:** Inventory reservation and payment response.
- **Asynchronous:** Order events, notifications and background processing.

3. ADR-003: Optimistic vs Pessimistic Concurrency

Decision: Use atomic inventory updates with strong concurrency control

Why: A flash sale creates high contention for limited inventory. Atomic updates ensure that inventory never becomes negative and successful sales never exceed available stock.
Trade-off: Strong concurrency control may increase contention, but inventory correctness is more important.

4. ADR-004: Cache Strategy

Decision: Use Cache-Aside for read-heavy data.

- Cache product and catalogue information.
- Keep the inventory database as the source of truth.
- Do not rely on cached inventory values for final reservation decisions.

5. ADR-003: Optimistic vs Pessimistic Concurrency

Decision: Use atomic inventory updates with strong concurrency control.

Why: A flash sale creates high contention for limited inventory. Atomic updates ensure that inventory never becomes negative and successful sales never exceed available stock.
Trade-off: Strong concurrency control may increase contention, but inventory correctness is more important.

6.Final Architecture Decisions

Area	Selected Decision
Database	SQL
Communication	Hybrid Sync + Async
Concurrency	Atomic Strong Concurrency Control
Caching	Cache-Aside

Overall Decision: SALESTORM prioritizes inventory correctness, transaction consistency and reliability while using asynchronous processing and caching to achieve scalability**.**